

Daily Progress & Next Steps Plan

Constraint-Interaction Metric for Symbolic Regression

Biswajyoti Nath

Subinoy Nath

July 6, 2026

Part 1: Summary of Today's Work (Expression Generator)

Today's focus was on transforming the stochastic expression generator from a rough prototype into a mathematically rigorous, fully documented, and robust foundation.

- **Mathematical Rigor & Domain Safety:** Discovered that `sympify(evaluate=False)` silently parsed mathematically invalid expressions (e.g., division by zero). We implemented a two-pass system: parsing followed by a numeric probe (`_validate_domain`) that strictly filters out NaN, infinity, and complex domain violations.
- **Stratified Sampling:** The original pure stochastic process failed to consistently reach deep trees. We implemented `generate_stratified()` and `generate_at_depth()` to guarantee uniform representation across depth buckets, which is vital for unbiased Monte Carlo density estimation.
- **Complete Reproducibility:** Isolated the global Python random state into a local instance (`self.rng`) driven strictly by the seed defined in `config.yaml`. We also introduced strict statistics tracking to monitor rejection rates.
- **Formal Specifications:** Upgraded the informal design notes into a formal `DESIGN_CONTEXT.md` that mathematically defines the probability space, outlines strict module invariants, and establishes a reproducibility contract.
- **Gold-Standard Documentation & Testing:** Executed the `code-documenter` and `codebase-documenter` protocols. We injected executable `doctest` examples, generated `API.md` and `ARCHITECTURE.md`, and wrote a 16-test suite that passes with 100% success. The codebase was formatted entirely using `ruff`.

Part 2: Strict Plan for Tomorrow

With the `expr_generator` robust and fully documented, tomorrow's efforts will focus on implementing the **Constraint Evaluators** and the **Monte Carlo Estimator**.

Phase 1: Implement Constraints (`src/constraints.py`)

We will build pure, deterministic Python functions that adhere strictly to the invariants defined in `DESIGN_CONTEXT.md`.

- C1. **Structural Constraints:** Ensuring specific variable presence or polynomial shapes.
- C2. **Depth Constraints:** Filtering ASTs that exceed strict complexity limits.

- C3. **Operator Whitelists:** Validating that only subset operators (e.g., $\{+, *, \sin\}$) exist in the tree.
- C4. **Positivity Constraints:** Ensuring the domain of the expression is strictly non-negative.
- C5. **Dimensional Constraints:** Enforcing specific physical dimensional consistency on the AST leaves.

Invariant check: All constraint functions must not mutate the SymPy AST and must gracefully handle constants and nested edge-cases.

Phase 2: Implement Interaction Metric (`src/metric.py`)

Once constraints are ready, we will construct the Monte Carlo density estimator.

- **Ingestion:** Consume N stratified expressions from `expr_generator.py`.
- **Evaluation:** Calculate $\rho(C_i)$, $\rho(C_j)$, and the joint probability $\rho(C_i \wedge C_j)$ for all pairs.
- **Interaction Calculation:** Compute the final coefficient matrix $M(i, j) = \frac{\rho(C_i \wedge C_j)}{\rho(C_i)\rho(C_j)}$.
- **Statistical Rigor:** Implement bootstrap resampling ($B = 1000$) to attach 95% confidence intervals to every $M(i, j)$ coefficient.

Phase 3: Experiment Logging

Build the metadata logging pipeline to fulfill the Reproducibility Contract, ensuring that Git hashes, configurations, seeds, and library versions are snapshotted alongside the final Interaction Matrix outputs.